

Hoạt Động 2:

1. Sự khác biệt giữa chạy file .py và gõ trực tiếp trong REPL:

-File .py (Script mode):

Code được viết, lưu trữ lâu dài trong file và có thể tái sử dụng hoặc chạy lại nhiều lần.

Phải chạy toàn bộ chương trình từ trên xuống dưới.

Phải dùng lệnh `print()` thì kết quả mới hiển thị ra màn hình Terminal.

-Chế độ REPL (Interactive mode):

Code chạy ngay lập tức sau từng dòng lệnh vừa gõ (thích hợp để thử nhanh 1-2 dòng lệnh).

Tự động in kết quả của các biểu thức đơn giản mà không bắt buộc dùng `print()`.

Dữ liệu và lệnh sẽ mất hoàn toàn ngay khi tắt cửa sổ Terminal.

2. Khi nào nên dùng Jupyter Notebook (.ipynb) thay vì file .py thông thường?

-Nên dùng Jupyter Notebook khi:

Học tập & Học máy / Phân tích dữ liệu (Data Science): Cần chia nhỏ chương trình thành các ô (cell) để chạy và kiểm tra từng đoạn code độc lập mà không phải chạy lại toàn bộ file từ đầu.

Trực quan hóa dữ liệu: Cần hiển thị ngay lập tức các biểu đồ, đồ thị, hoặc bảng dữ liệu bên dưới dòng code.

Làm báo cáo / Ghi chú: Khi muốn kết hợp cả đoạn code Python lẫn văn bản giải thích (Markdown), công thức toán học và hình ảnh trong cùng một file duy nhất.

Hoạt Động 3.1:

1diem	Không hợp lệ	Tên biến không được bắt đầu bằng chữ số.
gia-tri	Không hợp lệ	Chứa dấu gạch ngang - (Python hiểu nhầm là phép trừ).
_tam_thoi	Hợp lệ	Bắt đầu bằng dấu gạch dưới _ và chỉ chứa chữ cái.
Diem_TB	Hợp lệ	Đúng cú pháp Python (nhưng chưa chuẩn style PEP8 do viết hoa chữ cái đầu).
class	Không hợp lệ	Trùng với từ khóa hệ thống (keyword) của Python.
so luong	Không hợp lệ	Chứa khoảng trắng giữa các chữ.
MAX_SPEED	Hợp lệ	Đúng cú pháp và chuẩn PEP8 cho Hằng số (UPPER_CASE).
diemTB	Hợp lệ	Đúng cú pháp Python (theo kiểu camelCase).
2024_data	Không hợp lệ	Bắt đầu bằng chữ số.
tong\$	Không hợp lệ	Chứa ký tự đặc biệt \$.

Bài tập 3.2 – Áp dụng PEP8

Chuẩn PEP8 yêu cầu:

Tên biến: Dùng snake_case (viết thường toàn bộ, các từ phân cách bằng dấu _).	Dùng snake_case (viết thường toàn bộ, các từ phân cách bằng dấu _).
Tên biến:	Dùng snake_case (viết thường toàn bộ, các từ phân cách bằng dấu _).

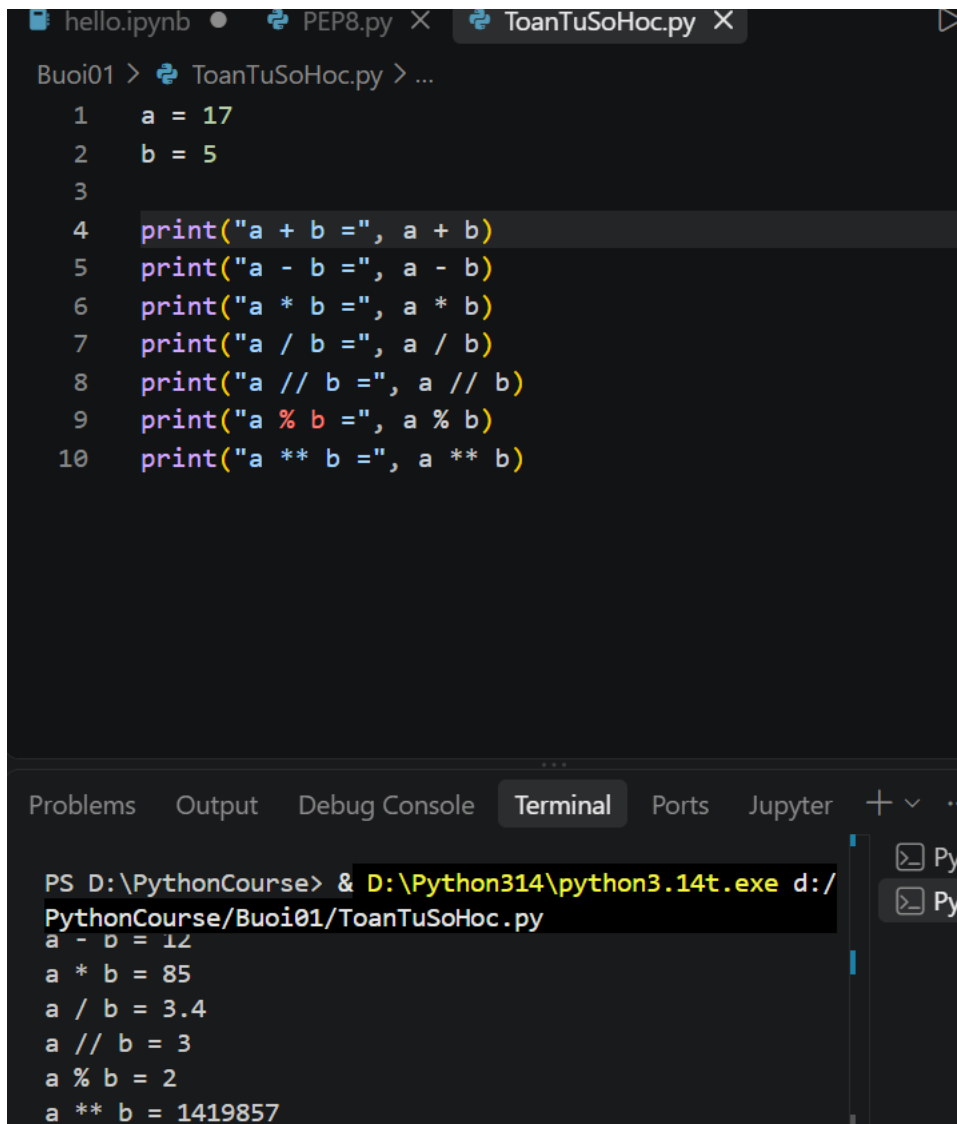
Viết lại code:

```
Buoi01 > PEP8.py > ...
1  ten = "Nguyen Van A"
2  diem_toan = 8.5
3  diem_van = 7.0
4  so_luong_mon_hoc = 2
5  MUC_LUONG_TOI_THIEU = 5000000 # Hằng số
6  print("--- THÔNG TIN SINH VIÊN ---")
7  print("Họ và tên:", ten)
8  print("Điểm Toán:", diem_toan)
9  print("Điểm Văn:", diem_van)
10 print("Số lượng môn học:", so_luong_mon_hoc)
11 print("Mức lương tối thiểu:", MUC_LUONG_TOI_THIEU)
```

Problems Output Debug Console Terminal Ports Jupyter + v ... | [] X

```
PS D:\PythonCourse> & D:\Python314\python3.14t.exe d:/
PythonCourse/Buoi01/PEP8.py
--- THÔNG TIN SINH VIÊN ---
Họ và tên: Nguyen Van A
Điểm Toán: 8.5
Điểm Văn: 7.0
Số lượng môn học: 2
Mức lương tối thiểu: 5000000
PS D:\PythonCourse>
```

Hoạt Động 5.1:



The screenshot shows a Jupyter Notebook interface with a file named `ToanTuSoHoc.py` open. The notebook contains 10 lines of Python code that define variables `a` and `b`, and then print the results of various arithmetic operations. Below the notebook, a terminal window is open, showing the command to run the script and its output.

```
Buoi01 > ToanTuSoHoc.py > ...  
1 a = 17  
2 b = 5  
3  
4 print("a + b =", a + b)  
5 print("a - b =", a - b)  
6 print("a * b =", a * b)  
7 print("a / b =", a / b)  
8 print("a // b =", a // b)  
9 print("a % b =", a % b)  
10 print("a ** b =", a ** b)
```

```
PS D:\PythonCourse> & D:\Python314\python3.14t.exe d:/PythonCourse/Buoi01/ToanTuSoHoc.py  
a - b = 12  
a * b = 85  
a / b = 3.4  
a // b = 3  
a % b = 2  
a ** b = 1419857
```

-Giải thích sự khác nhau:

+Giữa `/` và `//`:

~ `/` (Phép chia thường): Trả về kết quả thực đầy đủ dưới dạng số thực (float), ví dụ: $17 / 5 = 3.4$.

~ `//` (Phép chia lấy nguyên): Chỉ lấy phần nguyên của phép chia và loại bỏ phần thập phân, luôn trả về kết quả tròn xuống (int), ví dụ: $17 // 5 = 3$.

+Giữa `%` và `//`:

~ `%` (Phép chia lấy dư): Trả về phần dư còn lại của phép chia, ví dụ: $17 \% 5 = 2$ (vì 17 chia 5 được 3 dư 2).

~ `//` (Phép chia lấy nguyên): Trả về thương số nguyên của phép chia (lấy số lần chia hết), ví dụ: $17 // 5 = 3$.

Hoạt Động 5.4:

Dự đoán kết quả trước khi chạy:

```
print(2 + 3 * 4 ** 2)
```

Dự đoán: 50

```
print((2 + 3) * 4 ** 2)
```

Dự đoán: 80

```
print(10 > 5 and 3 < 1 or not False)
```

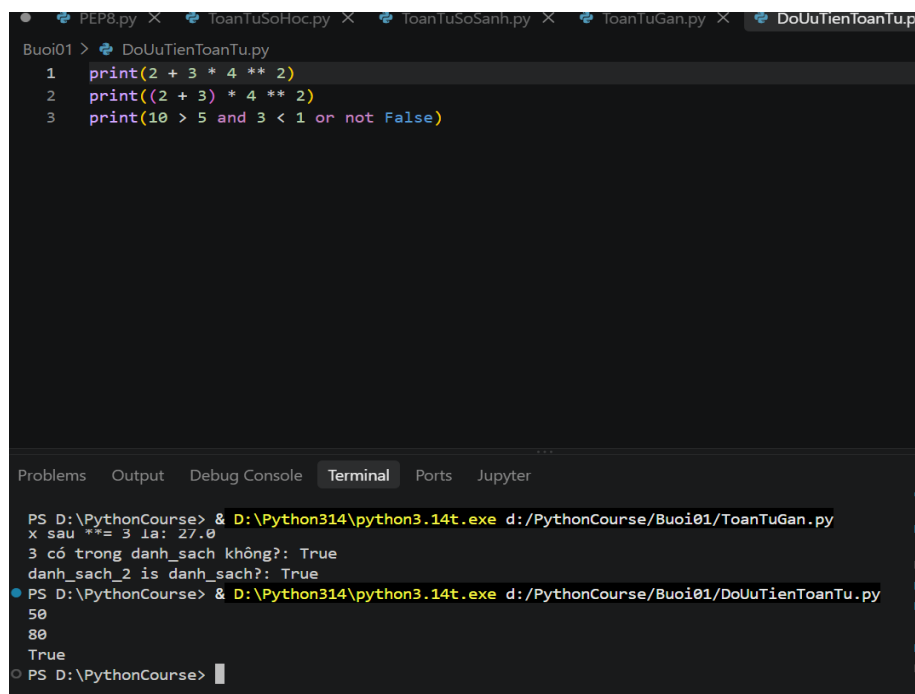
So sánh: (10 > 5) là True, (3 < 1) là False.not False trở thành True.

Biểu thức thành: True and False or True.

True and False thành False.

False or True thành True.

Dự đoán: True



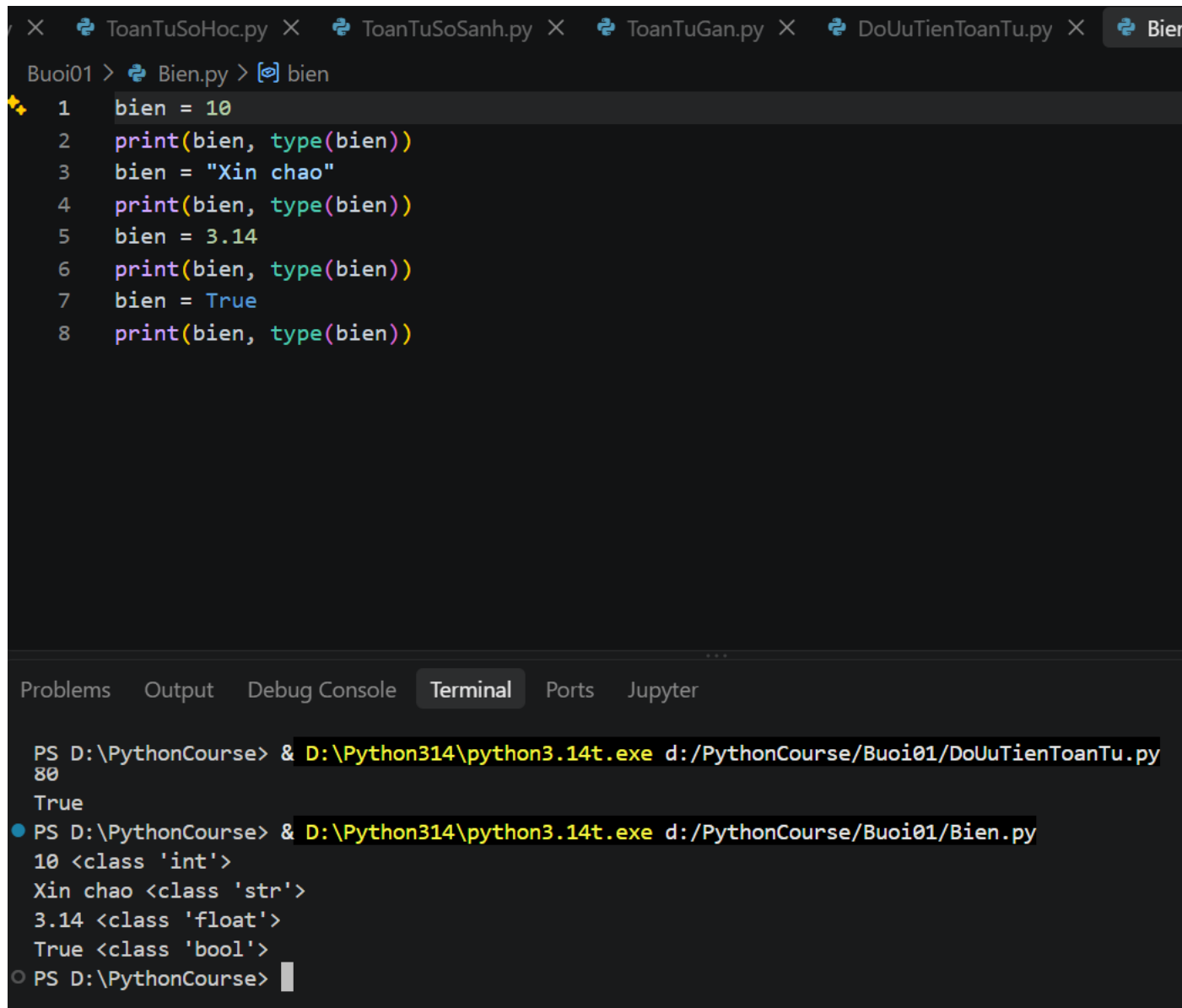
The screenshot shows a Python IDE with several tabs open: PEP8.py, toanTuSoHoc.py, toanTuSoSanh.py, toanTuGan.py, and DoUuTienToanTu.py. The active tab is DoUuTienToanTu.py, which contains the following code:

```
1 print(2 + 3 * 4 ** 2)
2 print((2 + 3) * 4 ** 2)
3 print(10 > 5 and 3 < 1 or not False)
```

The terminal window at the bottom shows the execution of the code. The first two lines of code are executed, resulting in the output 50 and 80. The third line of code is also executed, resulting in the output True.

```
PS D:\PythonCourse> & D:\Python314\python3.14t.exe d:/PythonCourse/Buoi01/ToanTuGan.py
x sau += 3 la: 27.0
3 có trong danh_sach không?: True
danh_sach_2 is danh_sach?: True
PS D:\PythonCourse> & D:\Python314\python3.14t.exe d:/PythonCourse/Buoi01/DoUuTienToanTu.py
50
80
True
PS D:\PythonCourse>
```

Hoạt Động 6.1:



```

ToanTuSoHoc.py X ToanTuSoSanh.py X ToanTuGan.py X DoUuTienToanTu.py X Bien
Buoi01 > Bien.py > bien
1 bien = 10
2 print(bien, type(bien))
3 bien = "Xin chào"
4 print(bien, type(bien))
5 bien = 3.14
6 print(bien, type(bien))
7 bien = True
8 print(bien, type(bien))

Problems Output Debug Console Terminal Ports Jupyter
PS D:\PythonCourse> & D:\Python314\python3.14t.exe d:/PythonCourse/Buoi01/DoUuTienToanTu.py
80
True
● PS D:\PythonCourse> & D:\Python314\python3.14t.exe d:/PythonCourse/Buoi01/Bien.py
10 <class 'int'>
Xin chào <class 'str'>
3.14 <class 'float'>
True <class 'bool'>
○ PS D:\PythonCourse>

```

-Dynamic Typing (Kiểu dữ liệu động): Python kiểm tra và xác định kiểu dữ liệu tại thời điểm thực thi (runtime) chứ không phải khi biên dịch.

-Cơ chế tham chiếu (Reference Model): Trong Python, biến không chứa giá trị trực tiếp mà chỉ đóng vai trò như một "nhãn" (label) tham chiếu đến một đối tượng trong bộ nhớ. Khi gán `bien = 10`, biến trỏ đến đối tượng số nguyên 10. Khi gán `bien = "Xin chào"`, nhãn biến chỉ đơn giản được gán lại để trỏ sang đối tượng chuỗi "Xin chào". Kiểu dữ liệu thuộc về đối tượng, không thuộc về tên biến.

- So sánh với C/C++/Java (Static Typing - Kiểu dữ liệu tĩnh): Trong C/C++/Java, biến phải được khai báo cố định kiểu dữ liệu trước (ví dụ `int bien = 10;`). Ở nhớ gán cho biến đó bị giới hạn chỉ chứa kiểu `int`, nếu gán chuỗi ký tự hay số thực vào sẽ báo lỗi biên dịch (Compile-time Error).

